MEMORIA TÉCNICA

Acortador de Enlaces

Una solución de Edge Computing sobre infraestructura Cloudflare

Autor:	**Jon Aldekoa**
Fecha:	**Abril 2026**
Repositorio:	github.com/Jaldekoa/acortador-enlaces
Demo:	acortador-enlaces.pages.dev

1 Introducción y Contexto

1.1 Antecedentes y Definición del Problema

En el ecosistema digital actual, compartir enlaces largos y complejos en medios de comunicación limitados —redes sociales, mensajería instantánea, presentaciones— supone una fricción innecesaria para el usuario. Las URLs generadas por plataformas modernas incluyen parámetros de seguimiento, tokens y rutas profundas que las hacen ilegibles, difíciles de recordar y propensas a errores de copia.

Los problemas concretos que motivaron este proyecto son:

- **Ilegibilidad:** Una URL de comercio electrónico típica puede superar los 200 caracteres, siendo inviable en canales de texto limitado.
- **Ineficiencia de almacenamiento:** Los sistemas que generan un nuevo registro por cada petición, sin verificar si la URL ya existe, producen bases de datos con miles de entradas duplicadas.
- **Falta de analítica básica:** La mayoría de soluciones caseras no registran el número de clics, imposibilitando medir el impacto de los enlaces.

1.2 Estado del Arte y Justificación

Antes de abordar el desarrollo, se analizaron las soluciones existentes:

- **Servicios SaaS (Bitly, TinyURL):** Ofrecen la funcionalidad de forma completa, pero no permiten control sobre la infraestructura ni sobre los datos. Además, sus planes gratuitos imponen límites de creación y los dominios no son personalizables.
- **Soluciones con servidor propio (Node.js + Redis/PostgreSQL):** Requieren gestión de servidores, configuración de escalado y costes de infraestructura, lo que resulta desproporcionado para una herramienta de utilidad personal.
- **Implementaciones con Cloudflare KV:** Almacenan pares clave-valor, pero carecen de capacidad de consulta SQL, lo que impide verificar duplicados por valor o filtrar por estado (active).

Nuestra propuesta: El Acortador de Enlaces resuelve estas limitaciones mediante una arquitectura íntegramente serverless sobre Cloudflare. El Worker actúa como orquestador entre el frontend y D1, base de datos SQLite nativa en el Edge que permite consultas SQL con latencia mínima. La separación entre Pages (frontend estático) y Workers (lógica de negocio) garantiza escalabilidad y mantenibilidad sin gestión de infraestructura.

2 Análisis de Requisitos

Para garantizar la viabilidad del MVP y delimitar el alcance del proyecto, se formalizaron los requisitos antes de iniciar el desarrollo.

2.1 Requisitos Funcionales (RF)

- **RF-01 Acortamiento de URL:** El sistema debe aceptar una URL larga (validada con new URL()) y devolver un enlace corto en formato {WORKER_URL}/{slug}.
- **RF-02 Generación de Slugs Únicos**: El Worker debe generar códigos alfanuméricos de 6 caracteres (62^6 = ~56 mil millones de combinaciones posibles).
- **RF-03 Deduplicación:** Antes de insertar un nuevo registro, el sistema debe verificar si la URL larga ya existe en la base de datos. En caso afirmativo, devuelve el slug existente sin crear un duplicado.
- **RF-04 Redirección HTTP:** Al acceder al enlace corto, el Worker debe redirigir al usuario a la URL original mediante un código HTTP 302 (redirección temporal).
- **RF-05 Contador de Visitas (Hits):** Cada redirección debe incrementar en 1 el contador hits del registro correspondiente en D1.
- **RF-06 Control de Estado:** El sistema debe soportar un campo active para poder desactivar enlaces sin borrarlos de la base de datos.

2.2 Requisitos No Funcionales (RNF)

- **RNF-01 Latencia Mínima:** Las redirecciones deben procesarse en el PoP (Point of Presence) de Cloudflare más cercano al usuario, con un objetivo de <50 ms en Europa.
- **RNF-02 Escalabilidad**: La arquitectura serverless debe soportar picos de tráfico sin intervención manual. Cloudflare Workers escala automáticamente a millones de peticiones.
- **RNF-03 No Bloqueo en Analítica:** La actualización del contador hits no debe bloquear la redirección al usuario. Se resuelve mediante ctx.waitUntil().
- **RNF-04 CORS:** La API debe aceptar peticiones cross-origin para permitir el consumo desde el dominio de Pages, diferente al del Worker.
- **RNF-05 Integridad de Datos:** El campo short_url debe tener restricción UNIQUE en el esquema SQL para evitar colisiones de slugs.

3 Planificación y Gestión del Tiempo

Se estimó una dedicación de 4-5 horas efectivas diarias, aplicando un factor de seguridad del 20% sobre las estimaciones para cubrir imprevistos.

3.1 Desglose de Tareas y Estimaciones (Backlog)

Fase	Tarea	Descripción Técnica	Est.	Dependencias
Diseño	Esquema DB & API	Diseño de la tabla D1, endpoints del Worker y contratos JSON.	4h	–
Backend	Setup Worker	Configuración del Worker, gestión de CORS y preflight OPTIONS.	3h	Diseño DB
Backend	Lógica POST /api/shorten	Verificación de URL duplicada, generación de slug y escritura en D1.	6h	Setup Worker
Backend	Lógica GET /:slug	Búsqueda en D1, actualización de hits (waitUntil) y redirección 302.	4h	POST /shorten
Frontend	UI Base (index.html + CSS)	Maquetación Dark Mode, formulario de entrada y zona de resultado.	5h	Diseño
Frontend	script.js – Consumo API	Validación de URL, llamada fetch POST, renderizado del enlace corto.	4h	Endpoints Back
Cierre	QA & Fixes	Pruebas manuales de flujos críticos y corrección de bugs.	4h	Integración
Docs	README + Memoria	Redacción de documentación técnica y limpieza del repositorio.	3h	–

3.2 Cronograma de Trabajo (Gantt)

El desarrollo se planificó en 5 días laborables, con las fases de Backend y Frontend en paralelo una vez completado el diseño de la API. El diagrama siguiente refleja la hoja de ruta:

```
gantt
    title Cronograma - Acortador de Enlaces
    dateFormat  YYYY-MM-DD
    section Diseño
    Esquema DB & Contrato API     :done, 2026-04-07, 1d
    section Backend
    Setup Worker + CORS           :done, 2026-04-07, 0.5d
    POST /api/shorten             :done, 2026-04-08, 1d
    GET /:slug + hits             :done, 2026-04-09, 1d
    section Frontend
    UI Base (HTML/CSS Dark Mode)  :done, 2026-04-08, 1d
    script.js (consumo API)       :done, 2026-04-09, 0.5d
    section Cierre
    QA & Fixes                   :done, 2026-04-10, 0.5d
    Documentación                :done, 2026-04-10, 0.5d
```

(Diagrama Mermaid renderizable en GitHub y herramientas compatibles)

4 Análisis Funcional y Flujos

El sistema se compone de dos flujos principales: el de acortamiento (creación) y el de redirección (uso). Ambos son gestionados íntegramente por el Worker, que actúa como única capa backend.

4.1 Diagrama de Secuencia (Flujo Completo)

A continuación, se detalla la interacción entre los cuatro componentes del sistema:

```
sequenceDiagram
    participant U  as Usuario
    participant FE as Frontend (Pages)
    participant W  as Cloudflare Worker
    participant DB as Cloudflare D1 (SQL)

    Note over U,DB: Proceso de Acortamiento
    U->>FE: Introduce URL larga
    FE->>W: POST /api/shorten { longURL }
    W->>DB: SELECT short_url WHERE long_url = ?
    alt URL ya existe
        DB-->>W: Devuelve slug existente
    else URL nueva
        W->>W: generateShortCode() → slug
        W->>DB: INSERT (short_url, long_url, hits:0, active:1)
    end
    W-->>FE: JSON { shortURL: WORKER_URL/slug }
    FE->>U: Muestra enlace acortado

    Note over U,DB: Proceso de Redirección
    U->>W: GET /slug
    W->>DB: SELECT long_url, active WHERE short_url = ?
    W->>DB: UPDATE hits = hits + 1 (waitUntil)
    W-->>U: HTTP 302 → URL original
```

(Diagrama Mermaid renderizable en GitHub y herramientas compatibles)

4.2 Lógica de Deduplicación del Worker

El Worker implementa la siguiente lógica de decisión antes de insertar cualquier registro nuevo:

- 1. Se recibe la URL larga en el cuerpo de la petición POST.
- 2. Se ejecuta SELECT short_url FROM table_links WHERE long_url = ? contra D1.
- 3. Si existe un registro, se devuelve el slug ya almacenado (sin escritura adicional).
- 4. Si no existe, se genera un nuevo slug de 6 caracteres y se ejecuta INSERT.

Este flujo garantiza que, independientemente del número de peticiones con la misma URL, la base de datos mantiene un único registro por destino.

5 Arquitectura y Modelo de Datos

5.1 Esquema de Base de Datos (D1 / SQLite)

El sistema utiliza una única tabla optimizada para lecturas frecuentes. El campo short_url tiene restricción UNIQUE para prevenir colisiones, y active permite gestionar el ciclo de vida del enlace sin borrado físico.

```
CREATE TABLE "main"."table_links" (
  "id"        INTEGER  NOT NULL,
  "short_url" TEXT     NOT NULL,   -- Slug generado (ej: 'aBcDe1')
  "long_url"  TEXT     NOT NULL,   -- URL de destino
  "hits"      INTEGER  DEFAULT 0,  -- Contador de visitas
  "active"    INTEGER  DEFAULT 1,  -- 1 = activo, 0 = desactivado
  PRIMARY KEY ("id")
);
```

Decisión de diseño: Se eligió SQLite (D1) sobre Cloudflare KV porque los datos son relacionales y requieren consultas por valor (long_url). KV solo permite consultas por clave, lo que imposibilitaría la deduplicación sin mantener un índice inverso separado.

5.2 Diseño de la API (Endpoints)

Método	Endpoint	Descripción	Auth
OPTIONS	/api/shorten	Preflight CORS. Devuelve 204 con headers de acceso.	No
POST	/api/shorten	Recibe { longURL }, devuelve { shortURL }. Deduplica en D1.	No
GET	/:slug	Busca el slug en D1, incrementa hits y redirige con HTTP 302.	No
GET	/	Ruta raíz: redirige al frontend (301).	No

5.3 Stack Tecnológico y Justificación

Tecnología	Rol	Justificación
HTML5 / CSS3 / JS Vanilla	Frontend	Sin framework para mantener el bundle en 0 KB. La interfaz es un formulario simple que no justifica React ni Vue.
Cloudflare Pages	Hosting Frontend	CDN global de Cloudflare con despliegue continuo desde Git. Latencia <30 ms en Europa para activos estáticos.
Cloudflare Workers	Backend / API	Edge Computing sin servidor. El código se ejecuta en el PoP más cercano al usuario, eliminando el cold start de una Lambda tradicional.
Cloudflare D1 (SQLite)	Base de Datos	SQL nativo en el Edge. Se eligió sobre KV porque los datos son relacionales (hits, estado) y D1 permite consultas SQL estándar con consistencia.
Wrangler CLI	Toolchain	Herramienta oficial de Cloudflare para configurar bindings, ejecutar dev local y desplegar el Worker.

6 Retos Técnicos y Soluciones

6.1 El Reto: Actualización de Hits sin Penalizar la Redirección

Situación: El requisito de analítica exige incrementar el contador hits en cada visita. Sin embargo, si la actualización de la base de datos se ejecuta de forma síncrona, el usuario experimenta el tiempo de escritura antes de recibir la redirección, penalizando la latencia percibida.

Problema: En un Worker estándar, una vez que se retorna la respuesta con Response.redirect(), la ejecución del contexto finaliza. Cualquier operación posterior se cancela.

Solución implementada: Se utilizó ctx.waitUntil() para registrar la promesa de escritura en D1 sin bloquear el envío de la respuesta al cliente. El Worker entrega el HTTP 302 inmediatamente y la plataforma Cloudflare mantiene el contexto activo hasta que la operación de base de datos se completa en background.

```
// La redirección se envía de inmediato al cliente
const updateHits = env.DB.prepare(
  'UPDATE table_links SET hits = hits + 1 WHERE short_url = ?'
).bind(slug).run();

ctx.waitUntil(updateHits); // Escritura en background

return Response.redirect(record.long_url, 302); // Sin esperar la escritura
```

Resultado: La latencia de redirección se reduce al tiempo de la lectura SELECT, desacoplando completamente la analítica del flujo crítico del usuario.

6.2 El Reto: Gestión de CORS en un Contexto Cross-Origin

Situación: El frontend se sirve desde acortador-enlaces.pages.dev y la API desde acortador-enlaces.jaldekoa.workers.dev. Los navegadores bloquean peticiones cross-origin sin las cabeceras CORS apropiadas. Además, las peticiones POST con Content-Type: application/json desencadenan un preflight OPTIONS previo.

Solución implementada: Se añadió un handler explícito para el método OPTIONS que devuelve HTTP 204 con las cabeceras Access-Control-Allow-Origin, Access-Control-Allow-Methods y Access-Control-Allow-Headers. Las cabeceras CORS se aplican a todas las respuestas JSON del Worker.

```
if (request.method === 'OPTIONS') {
  return new Response(null, {
    status: 204,
    headers: {
      'Access-Control-Allow-Origin': '*',
```

```
        'Access-Control-Allow-Methods': 'GET, POST, OPTIONS',
        'Access-Control-Allow-Headers': 'Content-Type'
      }
    });
  }
```

7 Pruebas de Calidad (QA)

Antes del cierre del proyecto se ejecutó una batería de pruebas manuales para validar los flujos críticos (Happy Path) y los casos límite (Edge Cases). Todas las pruebas se realizaron sobre el entorno de producción desplegado.

Caso de Prueba	Resultado Esperado	Estado
URL duplicada	Si se acorta la misma URL dos veces, el Worker devuelve el mismo slug, sin insertar un segundo registro.	✅ Pasado
URL inválida (sin http://)	El fetch lanza una excepción al hacer new URL(input). El frontend muestra el mensaje de error en rojo.	✅ Pasado
Slug inexistente	Al acceder a /xXxXxX con slug que no existe, el Worker devuelve 404 'Not found or inactive link'.	✅ Pasado
Enlace inactivo (active=0)	Si el registro existe pero active = 0, el Worker devuelve 404 en lugar de redirigir.	✅ Pasado
Contador de hits	Tras 3 visitas al enlace corto, el campo hits en D1 refleja el valor 3.	✅ Pasado
CORS preflight	Una petición OPTIONS recibe 204 con Access-Control-Allow-Origin: *.	✅ Pasado

8 Conclusiones y Futuros Pasos

El desarrollo del Acortador de Enlaces ha permitido consolidar conocimientos sobre arquitecturas serverless, Edge Computing y el paradigma de bases de datos distribuidas. El sistema es funcional, está desplegado en producción y cumple con todos los requisitos funcionales y no funcionales definidos.

Las principales conclusiones técnicas obtenidas son:

- El modelo de deduplicación mediante SELECT previo al INSERT reduce significativamente el crecimiento de la base de datos en escenarios de uso real, donde múltiples usuarios comparten las mismas URLs.
- ctx.waitUntil() es una herramienta fundamental en Workers para desacoplar operaciones de analítica del flujo crítico del usuario, sin necesidad de colas ni servicios externos.
- La separación entre Cloudflare Pages y Cloudflare Workers permite iterar de forma independiente sobre frontend y backend, aplicando el principio de separación de responsabilidades.

8.1 Mejoras Propuestas para la v2.0

- Panel de administración: Interfaz para visualizar todos los enlaces activos, sus hits y estado, con capacidad de activar/desactivar desde UI.
- Slugs personalizados: Permitir al usuario definir un slug personalizado en lugar del generado aleatoriamente.
- TTL (Time To Live): Añadir un campo expires_at para que los enlaces caduquen automáticamente tras un período configurable.
- Autenticación básica: Proteger el endpoint POST /api/shorten con un token de API para evitar uso no autorizado.
- Exportación de analítica: Endpoint GET /api/stats que devuelva los datos de hits en formato JSON o CSV.

Fin del documento.